Python Engineering - Reference Guide

Class 13 - Classes & Objects: referencia de sintaxis OOP

Esta guia es deliberadamente mas explicativa que las anteriores. El objetivo es construir un modelo mental claro de lo que ocurre cuando Python crea y usa objetos.

1. class

Que hace: define un nuevo tipo de objeto. Dentro de la clase describimos que datos guardara cada objeto y que operaciones podra realizar.
Que produce: una clase que luego podemos instanciar.

```
class VerticalState:
    ...
```

Piensa en la clase como el plano de diseno; todavia no es un estado fisico concreto.

2. Objeto / instancia

Un objeto es una realizacion concreta de una clase. La expresion VerticalState(...) crea una instancia.

```
earth_state = VerticalState(100.0, 20.0)
moon_state  = VerticalState(100.0, 20.0, g=1.62)
```

Ambos son VerticalState, pero cada uno tiene sus propios valores almacenados.

3. __init__()

Que hace: inicializa el objeto inmediatamente despues de crearlo.
Que recibe: self y los argumentos de inicializacion.
Que devuelve: normalmente no escribimos return.

```
def __init__(self, h, v, t=0.0, g=9.81):
    self.h = h
    self.v = v
    self.t = t
    self.g = g
```

En EX21, __init__ deja preparado el estado inicial de cada objeto.

4. self

Que significa: self es la referencia al objeto concreto sobre el que se esta ejecutando el metodo.

```
earth_state.specific_energy()
# self -> earth_state

moon_state.specific_energy()
# self -> moon_state
```

El mismo metodo puede trabajar con objetos distintos porque self identifica cual se esta usando.

5. Parametro h vs atributo self.h

En __init__(self,h,...), h es un valor recibido temporalmente. self.h queda almacenado dentro del objeto.

```
self.h = h
# h      -> incoming parameter
# self.h -> stored attribute
```

El parametro desaparece al terminar la llamada; el atributo pertenece al objeto.

6. Atributo

Un atributo es un dato asociado a un objeto. En EX21: self.h, self.v, self.t y self.g.

```
earth_state.h
earth_state.v
earth_state.g
```

Los atributos describen el estado del objeto.

7. Metodo

Un metodo es una funcion definida dentro de una clase y pensada para trabajar con el objeto. Normalmente recibe self como primer parametro.

```
def specific_energy(self):
    return self.g * self.h + 0.5 * self.v**2
```

Entrada explicita: ninguna. Entrada implicita: self. Salida: energia especifica.

8. Metodo que modifica el estado

euler_step(dt) no solo calcula: cambia h, v y t del objeto. Esto se llama mutar o actualizar el estado.

```
earth_state.euler_step(0.1)
# earth_state now stores new h, v and t
```

Despues de la llamada, el mismo objeto contiene valores diferentes.

9. isinstance()

Que hace: comprueba si un objeto pertenece a una clase o tipo.
Que devuelve: True o False.

```
isinstance(earth_state, VerticalState)
# True
```

Es util para inspeccion y validacion de tipos.

10. vars() como herramienta de aprendizaje

Para objetos sencillos, vars(objeto) muestra el diccionario interno de atributos de instancia.

```
vars(earth_state)
# {'h':100.0, 'v':20.0, 't':0.0, 'g':9.81}
```

No es necesario para resolver EX21; sirve para visualizar que guarda el objeto.

11. Llamada mediante objeto vs llamada mediante clase

Cuando llamamos un metodo desde el objeto, Python proporciona self automaticamente.

```
earth_state.specific_energy()
# usual form

VerticalState.specific_energy(earth_state)
# self supplied explicitly
```

Este ejemplo explica por que self aparece en la definicion aunque no lo escribamos al llamar mediante el objeto.

12. Resumen mental

class = define el tipo o plano de diseno.
object = instancia concreta.
__init__ = configura los atributos iniciales.
self = referencia al objeto actual.
attribute = dato almacenado en el objeto.
method = funcion asociada al objeto.
mutation = cambio del estado interno del objeto.